NEXT WORD PREDICTION SYSTEM

Using Deep Learning, RNN & LSTM

Project Report Submitted
In partial fulfilment of the requirement for the degree of

Bachelor of Technology (B.Tech) | 2023–2027

Computer Science Engineering (in spl. With Artificial Intelligence & Machine Learning)

By

Ujjwal Tripathi (2301730143)
Yash Vashisth (2301730149)
Gopesh Aggarwal (2301730158)
Aditya Singh (2301730138)
Nitesh Kumar Mandal (2301730139)

Under the Guidance of
Mr. Kunal Rai

SOET

K. R. Mangalam University, Gurugram – 122003

April 2026

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all the principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

We understand that any violation of the above will cause disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

We further declare that if any violation of the intellectual property right or copyright, our supervisor and university should not be held responsible for the same.

Student Name	Roll No.	Signature
Ujjwal Tripathi	2301730143	
Yash Vashisth	2301730149	
Gopesh Aggarwal	2301730158	
Aditya Singh	2301730138	
Nitesh Kumar Mandal	2301730139	

Place: K.R. Mangalam University

Date: 29 April 2026

CERTIFICATE

This is to certify that the project report titled "Next Word Prediction System using Deep Learning, RNN & LSTM" has been submitted by the following students as part of their coursework for the degree of Bachelor of Technology (B.Tech) in Computer Science Engineering (AI & ML) at K.R. Mangalam University.

Name of Student	Roll Number
Ujjwal Tripathi	2301730143
Yash Vashisth	2301730149
Gopesh Aggarwal	2301730158
Aditya Singh	2301730138
Nitesh Kumar Mandal	2301730139

This work has been carried out under our supervision and has not been submitted anywhere else for any other degree or examination.

Name of Supervisor: Mr. Kunal Rai

Date: 29 April 2026

Place: K.R. Mangalam University

ACKNOWLEDGEMENT

We would like to take a moment to sincerely thank everyone who supported us throughout this project. Working with Deep Learning frameworks and NLP pipelines was both challenging and rewarding, and we could not have done it without the right guidance.

We are truly grateful to our supervisor, Mr. Kunal Rai, for his constant support, constructive feedback, and for always being available when we faced difficulties. His insights on model architecture and training significantly shaped the outcome of this project.

We also extend our gratitude to K.R. Mangalam University and the School of Engineering & Technology (SOET) for providing the resources, infrastructure, and academic environment that made this work possible.

A heartfelt thank you to our friends and classmates who helped us brainstorm solutions, debug code, and stay motivated throughout the process.

Place: K.R. Mangalam University

Date: 29 April 2026

Ujjwal Tripathi (2301730143)

Yash Vashisth (2301730149)

Gopesh Aggarwal (2301730158)

Aditya Singh (2301730138)

Nitesh Kumar Mandal (2301730139)

EXECUTIVE SUMMARY / ABSTRACT

Language is one of the most complex aspects of human intelligence. With the rapid growth of digital communication, there is an increasing demand for intelligent systems that can predict and complete human text. Next Word Prediction is a foundational Natural Language Processing (NLP) task that underlies technologies such as autocomplete, smart keyboards, and AI writing assistants.

This project presents a Next Word Prediction System built using Deep Learning techniques — specifically Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks. The model was trained on the complete text of Pride and Prejudice by Jane Austen, sourced from Project Gutenberg. The training dataset was tokenized using Keras's Tokenizer, and sequential 4-word windows were used to create input-output pairs for model training.

The model architecture consists of an Embedding layer, two stacked LSTM layers (each with 500 units), followed by two Dense layers with ReLU and Softmax activations respectively. The model was compiled with categorical cross-entropy loss and trained using the Adam optimizer over 30 epochs with a batch size of 64.

A Streamlit web application was developed to allow users to enter any sequence of words and receive a predicted next word in real time. The trained model and tokenizer were saved and loaded for inference, enabling seamless deployment.

Key Words: Deep Learning, Recurrent Neural Network, LSTM, Natural Language Processing, Tokenization, Embedding, Streamlit, Next Word Prediction.

1. INTRODUCTION

1.1 Background

We live in the age of intelligent text — from autocomplete on smartphones to AI writing assistants, next word prediction is everywhere. These systems rely on understanding context: what words came before and what is most likely to come next. This project builds that capability from scratch using Deep Learning.

Recurrent Neural Networks (RNNs) were designed specifically for sequential data. However, standard RNNs suffer from the vanishing gradient problem, making them poor at capturing long-range dependencies in text. Long Short-Term Memory (LSTM) networks solve this by introducing gating mechanisms — input gate, forget gate, and output gate — that allow the model to selectively remember and forget information over long sequences.

In this project, we train a two-layer stacked LSTM model on Pride and Prejudice, a classic English novel with rich vocabulary and complex sentence structures. The model learns the statistical patterns of the English language from this corpus and uses them to predict the next word given any three-word input sequence.

1.2 Problem Statement

Modern Natural Language Processing demands systems that understand sequential context and generate linguistically plausible predictions. While large pre-trained models like GPT-4 or BERT achieve impressive results, their training requires massive compute resources and datasets. The challenge addressed in this project is: can we build a reasonably accurate next word prediction model using a single literary text and a custom LSTM architecture, trained from scratch?

The specific challenge is to train an LSTM model that, given any three-word input from the training vocabulary, correctly predicts the most probable fourth word — capturing both short and medium-range linguistic dependencies present in the training text.

1.3 Objectives

Primary Objective

To design, train, and deploy a Deep Learning model that predicts the next word in a sequence using LSTM-based architecture trained on a real literary corpus.

Specific Objectives

- Collect and preprocess a raw text corpus (Pride and Prejudice) for NLP model training.
- Implement tokenization and convert text into integer sequences using Keras Tokenizer.
- Create n-gram based input-output pairs using a sliding window of 4 words.
- Design and train a stacked LSTM model with Embedding, Dense, and activation layers.
- Save the trained model and tokenizer for reuse and deployment.
- Build and deploy a Streamlit web application for interactive next-word prediction.
- Evaluate model behaviour and discuss limitations and future improvements.

2. DATA DESCRIPTION & TECHNICAL STACK

2.1 Data Description

The training data for this project is the complete text of Pride and Prejudice by Jane Austen, sourced from Project Gutenberg (www.gutenberg.org) as a plain text file. This novel contains approximately 122,000 words across 61 chapters, providing a rich and varied vocabulary suitable for training a language model.

The text was preprocessed to remove newline characters, carriage returns, Unicode characters, and special quotation marks. All whitespace was normalized and the resulting clean string was used as the sole input to the tokenizer.

Feature	Description	Data Type
Raw Text	Full text of Pride and Prejudice	String
Tokens	Integer-encoded word sequence from tokenizer	Integer Array
Input (X)	Sequences of 3 consecutive token IDs	NumPy Array
Output (y)	4th token ID — the target word to predict	One-Hot Encoded
Vocabulary Size	Total unique tokens + 1 (for index offset)	Integer

2.2 Technical Stack

Purpose	Tools & Libraries
Programming Language	Python 3.x
Deep Learning Framework	TensorFlow 2.15.0, Keras
Model Layers	Embedding, LSTM, Dense

Data Processing	NumPy, Keras Tokenizer
Model Serialization	model.save() (.h5 / .keras), Pickle (.pkl)
Web Application	Streamlit
Development Environment	Jupyter Notebook / Google Colab

2.3 Data Source

The dataset is Pride and Prejudice (1813) by Jane Austen, sourced from Project Gutenberg (https://www.gutenberg.org). The text is in the public domain and freely available. No additional external dataset was used. The entire model is trained on this single literary corpus, making it a focused, domain-specific language model.

3. EXPLORATORY DATA ANALYSIS (EDA)

3.1 Data Cleaning

Before any modelling, the raw text underwent the following preprocessing steps:

- Removed newline characters (\n), carriage returns (\r), and Unicode markers (\ufeff) that appear in Project Gutenberg files.
- Stripped special quotation marks that would be treated as separate tokens.
- Split text into a list of words using Python's .split() method and rejoined with single spaces to normalize whitespace.
- Passed the cleaned text string to Keras Tokenizer, which further lowercased all words and indexed the vocabulary.
- Saved the tokenizer object using Pickle so the same word-to-index mapping can be reused during prediction.

3.2 Descriptive Statistics

After tokenization and sequence generation, the following statistics were observed:

Statistic	Value
Total word tokens in corpus	~122,000
Unique vocabulary size	~6,970 words
Total training sequences (4-grams)	~121,997
Input shape (X)	(121997, 3)
Output shape (y)	(121997, vocab_size) — one-hot
Sequence window size	4 words (3 input + 1 target)

3.3 Sequence Generation Strategy

A sliding window approach was used to generate training sequences. For every position i starting from index 3 in the token sequence, the model uses tokens at positions [i-3, i-2, i-1] as inputs (X) and the token at position

[i] as the label (y). This produces overlapping 4-gram sequences across the entire corpus, maximizing training data from the single source text.

The output labels were one-hot encoded using Keras's to_categorical() function, producing a sparse binary vector of length equal to the vocabulary size for each training example.

3.4 Data Visualization

The following visual analyses were performed to understand the data:

- Vocabulary Frequency Distribution: A histogram of word frequencies showed a steep power-law distribution — a small number of common words (the, and, to, of) account for a large proportion of the corpus, consistent with Zipf's Law.
- Sequence Count Verification: A count of total sequences confirmed alignment with corpus length minus the window size.
- Token Index Distribution: Visualization of token index distribution showed that lower indices (most frequent words) are heavily represented.
- Training Loss Curve: After training, plotting the categorical cross-entropy loss per epoch showed a consistent decline, confirming successful model convergence.

4. MODEL DEVELOPMENT & ANALYSIS

4.1 Feature Engineering

Rather than raw text, the model operates on tokenized integer sequences. The Keras Tokenizer creates a word index (a dictionary mapping each unique word to a unique integer). The text is then converted to a list of integers, and 4-word sliding windows extract (X, y) pairs. The Embedding layer at the start of the model transforms each integer token into a dense 10-dimensional vector, allowing the model to learn semantic relationships between words during training.

4.2 Complete Model Code

The following section presents the complete implementation code, organized cell-by-cell as written in the Jupyter Notebook.

Cell 1 — Install Dependencies

```
# Cell 1: Install dependencies (run once)
!pip install tensorflow==2.15.0
```

Cell 2 — Import Libraries

```
# Cell 2: Import Libraries
import tensorflow as tf
from tensorflow.keras.preprocessing.text import
Tokenizer
from tensorflow.keras.layers import Embedding,
LSTM, Dense
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
import pickle
import numpy as np
import os
```

Cell 3 — Load and Pre-process the Text Data

```
# Cell 3: Open and pre-process the text data
```

```
file = open("Pride and Prejudice.txt", "r",
encoding="utf8")

lines = []
for i in file:
    lines.append(i)

data = " ".join(lines)

# Remove unnecessary characters
data = data.replace('\n', '').replace('\r', '')
         .replace('\ufeff', '')
         .replace('\u201c', '').replace('\u201d',
'')

# Normalize whitespace
data = data.split()
data = " ".join(data)
print(data[:500])   # preview first 500
characters
```

Cell 4 — Tokenization & Vocabulary Building

```
# Cell 4: Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])

# Save tokenizer for prediction function
pickle.dump(tokenizer, open('token.pkl', 'wb'))

sequence_data =
tokenizer.texts_to_sequences([data])[0]

vocab_size = len(tokenizer.word_index) + 1
print('Vocabulary Size:', vocab_size)
```

Cell 5 — Generate N-gram Sequences (Sliding Window)

```
# Cell 5: Create n-gram sequences (sliding window
of 4)
sequences = []
```

```
for i in range(3, len(sequence_data)):
    words = sequence_data[i-3:i+1]
    sequences.append(words)

print('Total sequences:', len(sequences))
sequences = np.array(sequences)

# Split into inputs (X) and labels (y)
X = []
y = []
for i in sequences:
    X.append(i[0:3])   # first 3 words
    y.append(i[3])     # 4th word = target

X = np.array(X)
y = np.array(y)

# One-hot encode the labels
y = to_categorical(y, num_classes=vocab_size)
```

Cell 6 — Build the Stacked LSTM Model

```
# Cell 6: Build the LSTM Model
model = Sequential()
model.add(Embedding(vocab_size, 10,
input_length=3))
model.add(LSTM(500, return_sequences=True))
model.add(LSTM(500))
model.add(Dense(1000, activation='relu'))
model.add(Dense(vocab_size,
activation='softmax'))
model.summary()
```

Cell 7 — Compile and Train the Model

```
# Cell 7: Compile and Train the Model
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

```
model.fit(X, y, epochs=30, batch_size=64)

# Save trained model in both formats
model.save('next_word_model.h5')
model.save('next_word_model.keras')
```

Cell 8 — Load Model & Prediction Function

```
# Cell 8: Prediction Function
from tensorflow.keras.models import load_model

model     = load_model('next_word_model.h5')
tokenizer = pickle.load(open('token.pkl', 'rb'))

def Predict_Next_Words(model, tokenizer, text):
    sequence =
tokenizer.texts_to_sequences([text])
    sequence = np.array(sequence)
    preds    = np.argmax(model.predict(sequence))
    predicted_word = ''
    for key, value in
tokenizer.word_index.items():
        if value == preds:
            predicted_word = key
            break
    print(predicted_word)
    return predicted_word
```

Cell 9 — Interactive Prediction Loop

```
# Cell 9: Interactive Prediction Loop
while True:
    text = input("Enter your line: ")
    if text == "0":
        print("Execution completed.....")
        break
    else:
        try:
            text = text.split(" ")
```

```
            text = text[-3:]    # take last 3
words
            print(text)
            Predict_Next_Words(model, tokenizer,
text)
        except Exception as e:
            print("Error occurred: ", e)
            continue
```

4.3 Model Architecture Summary

The model was designed using Keras Sequential API with the following layer stack:

Layer	Type	Configuration
1	Embedding	vocab_size inputs → 10-dim vectors, input_length=3
2	LSTM	500 units, return_sequences=True
3	LSTM	500 units
4	Dense	1000 units, activation=ReLU
5	Dense (Output)	vocab_size units, activation=Softmax

The two stacked LSTM layers allow the model to capture both lower-level sequential patterns (Layer 1) and higher-level temporal abstractions (Layer 2). The large Dense layer (1000 units) with ReLU activation acts as a non-linear classifier before the final Softmax output, which produces a probability distribution over the entire vocabulary.

4.4 Training Configuration & Performance

Parameter	Value
Loss Function	Categorical Cross-Entropy
Optimizer	Adam

Epochs	30
Batch Size	64
Model Export	.h5 (legacy) and .keras (recommended)

Training loss decreased consistently across all 30 epochs, indicating successful gradient descent convergence. Qualitative evaluation showed that predictions were linguistically plausible — for example, input 'in the United' correctly predicted 'States', consistent with frequent phrases in the training text.

5. RESULTS & DISCUSSION

5.1 Key Insights

- LSTM Effectiveness: The stacked LSTM architecture successfully captured sequential dependencies in English text, producing contextually appropriate word predictions from the literary corpus.
- Embedding Layer Value: The trainable embedding layer allowed the model to develop its own dense word representations, adapting to the specific vocabulary and style of Pride and Prejudice.
- Training Corpus Impact: Training on a single literary novel produced a model biased toward 19th-century English phrasing and vocabulary. While this limits generalization, it demonstrates the viability of domain-specific language models.
- Prediction Behaviour: The model consistently predicted grammatically valid words. For structured inputs found in the training text, predictions were highly accurate.
- Scalability: The same architecture and code pipeline can be extended to larger corpora, more LSTM layers, bidirectional LSTMs, or Transformer-based alternatives with minimal code changes.

5.2 Streamlit Application

A web-based interface was built using Streamlit to make the model accessible to non-technical users. The application:

- Displays the project title and a descriptive banner image.
- Accepts user input as a text box (minimum 3 words required).
- Loads the saved model (.h5) and tokenizer (.pkl) for inference.
- Extracts the last 3 words from the input, passes them through the model, and displays the predicted next word.
- Provides immediate real-time feedback with a 'Find the next word' button.

6. CONCLUSION & RECOMMENDATIONS

6.1 Summary of Findings

This project successfully designed and trained a Next Word Prediction System using a two-layer stacked LSTM deep learning model. Trained entirely on the Pride and Prejudice corpus, the model demonstrated the capability to predict contextually and grammatically appropriate next words given any three-word input sequence. The project confirmed that:

- LSTM networks are well-suited to sequential text modelling tasks and can effectively learn word-level language patterns from a single corpus.
- Keras Tokenizer combined with one-hot encoding provides a straightforward and effective pipeline for training word-level language models.
- Stacked LSTM layers with a large Dense intermediate layer improve the model's ability to capture both low-level and high-level linguistic patterns.
- A Streamlit front-end makes the model immediately accessible and demonstrates a practical deployment path for DL-based NLP applications.

6.2 Strategic Recommendations

- Larger and Diverse Corpus: Training on a broader, more modern text corpus would significantly improve vocabulary coverage and generalization.
- Bidirectional LSTM: Replacing unidirectional LSTMs with Bidirectional LSTMs would allow the model to leverage both past and future context.
- Beam Search Decoding: Implementing beam search would allow the model to generate multiple candidate next words and select the most probable sequence.

- Transformer Architecture: Replacing the LSTM stack with a Transformer encoder (BERT, GPT-2) would dramatically improve performance on long-range dependencies.
- Mobile & API Deployment: Exporting the model to TensorFlow Lite or a Flask REST API would enable integration into mobile keyboard applications.

6.3 Future Scope

- Multilingual Support: Extending the model to predict words in Hindi, French, or other languages by training on multilingual corpora.
- Top-K Predictions: Returning the top-3 or top-5 most probable next words to give users multiple intelligent suggestions.
- Fine-tuning Pre-trained Models: Using a pre-trained GPT-2 or DistilBERT model and fine-tuning on a custom dataset for significantly higher accuracy.
- Real-Time Cloud Deployment: Hosting the Streamlit app on Streamlit Cloud or Hugging Face Spaces for public accessibility.
- Context Window Expansion: Increasing the input window from 3 words to 5 or 10 words to capture longer-range contextual dependencies.

7. REFERENCES & APPENDIX

7.1 Bibliography

- Project Gutenberg — Pride and Prejudice by Jane Austen: https://www.gutenberg.org/ebooks/1342

- TensorFlow & Keras Official Documentation — https://www.tensorflow.org/

- Streamlit Official Documentation — https://docs.streamlit.io/

- Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

- Sundermeyer, M., Schluter, R., & Ney, H. (2012). LSTM Neural Networks for Language Modeling. Interspeech.

- Chollet, F. (2021). Deep Learning with Python, 2nd Edition. Manning Publications.